

C++ Programming

Trainer : Rajiv K

Email: rajiv.kamune@sunbeaminfo.com



Agenda

- **dynamic memory allocation**
- **copy constructor**
- **shallow copy**
- **deep copy**
- **friend function**
- **Operator overloading**



dynamic memory allocation

- Dynamic memory allocation in C/C++ refers to performing **memory allocation manually by a programmer**.
- Dynamically allocated memory is allocated **on Heap**,
- It is the **programmer's responsibility to deallocate** memory **when no longer needed**.
- In C++, to **allocate memory** dynamically we should use **new operator** and to **deallocate** that **memory** we should use **delete operator**.
- If **new operator fails to allocate** memory then C++ runtime implicitly throws **bad_alloc exception**.
- We can also **initialize** the memory for **built-in data types** using a new operators as well as **user defined data types** including **structure** and **class**, a constructor is get called.



new operators

new operator:

- The new operator denotes a **request** for **memory allocation**.
- A **new** operator **allocate the memory** and **returns** the **address** of the newly allocated and initialized memory **to the pointer variable**.

Syntax to use new operator:

data-type pointer-variable= new data-type;

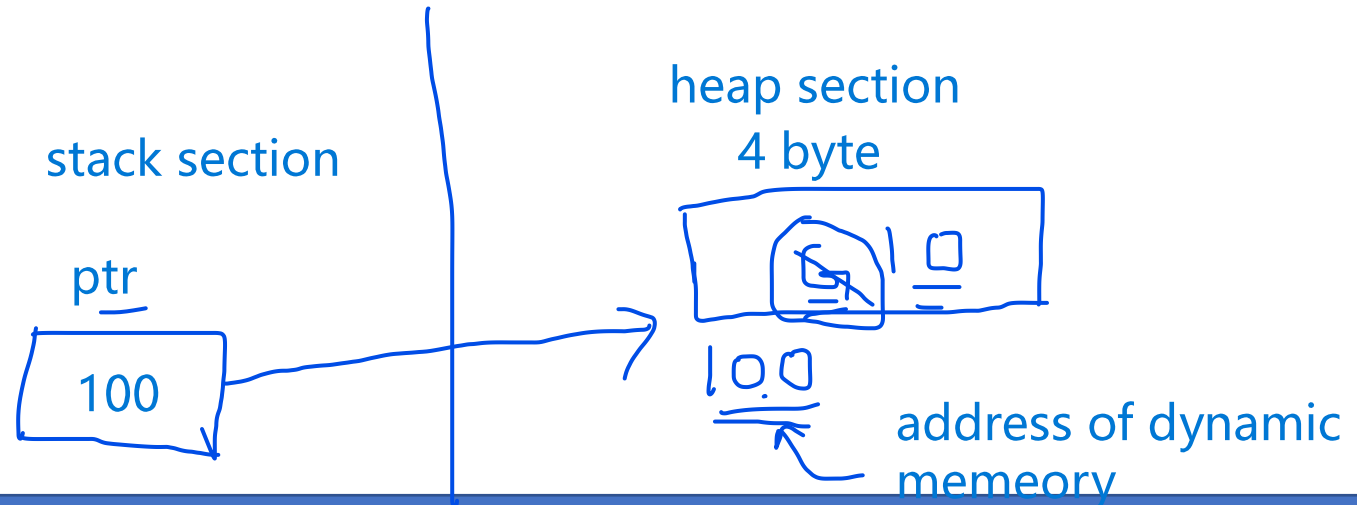
Example:

`int* ptr=new int;` 4 byte

`*ptr=10;`

value at address(100)=10

`int * ptr=new int(10);`



Examples

Point* ptr = new Point; //parameterless constructor will call

Point* ptr = new Point(); //parameterless constructor will call
//Recommended

Point* ptr = new Point(10, 20); //parameterized constructor will call

delete ptr;

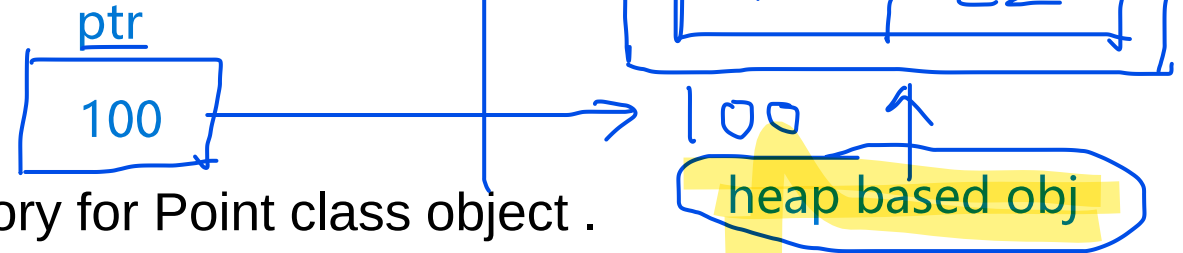
//destructor will call

int * ptr = new int(10);

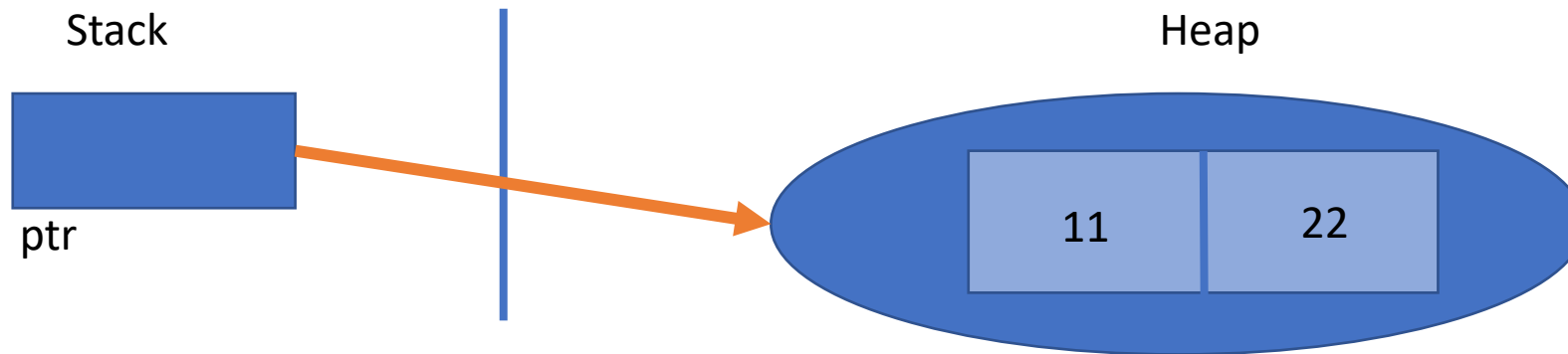


Heap Based object

```
Point *ptr=new Point(11,22) ;  
delete ptr;
```



- By using `new` we are allocating dynamic memory for Point class object .
- Object get created on heap section hence this object is call Heap based or dynamic object.
- `ptr` is point type pointer which is holding the address of that dynamic object.



memory block(Array) memory allocation using new

- To allocate the dynamic memory for **5 continuous** of type double use following example to under stand it better.
- `double* ptr=new double[5];`
- It would returns a pointer to the first element of the sequence.
- In above diagram `ptr[0]` would give first element, `ptr[1]` would give second element so on and so forth.



delete operator

- In C++ programmer can **delete** the **dynamically** allocated **memory** using **delete operator**.

Syntax:

```
delete pointer-variable;
```

Example:

```
int * ptr=new int; //Dynamic memory allocation
```

```
delete ptr;           //deleting the dynamically allocated  
                      //memory using delete operator
```

```
int * pqr=new int[5]; // Dynamically allocating the memory for block
```

```
delete[ ] pqr; // deleting the block of memory which is been  
              //allocated the memory dynamically.
```



delete operator

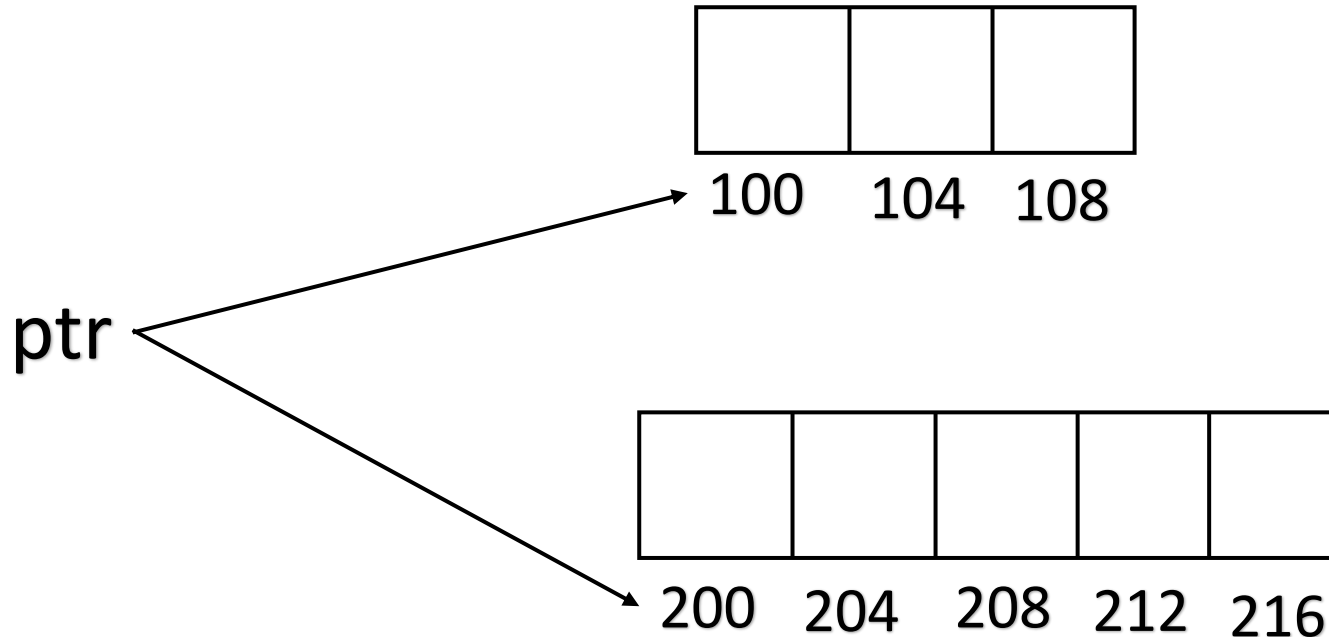
- **Memory leakage:** If we reserve space in memory but no pointer is pointing to it then **wastage of such memory** is called **memory leakage**.

- **Example:**

```
int* ptr = new int[3];
```

```
ptr = new int[5];
```

//this statement definitely loose 12 bytes.



Copy Constructor

- A Copy constructor is parameterized constructor of the class used to declare and initialize an object from another object.
- It takes single parameter of same type as reference.
- If developer **dose not write the copy constructor within the class** then in that case **compiler provides the copy constructor** when ever it is required.
- **Syntax:**

```
class_name(const class_name & variable_name)           //Copy constructor
{
    //Body of copy constructor
}
```

Point p1; // x=10; y=20 existing object

Point p2=p1; //copy construc



Copy Constructor

- **Copy Constructor Gets called in following Conditions.**

1. If we pass object to the function as value.
2. If we return object from function as value.
3. If we initialize object from existing object.
4. If we throw object
5. If we catch object

```
try
{
    throw 10;
    throw new Pont; //copy con will be called
}
catch(int ex){}
catch(Popint p1){} //copy con will be called
```

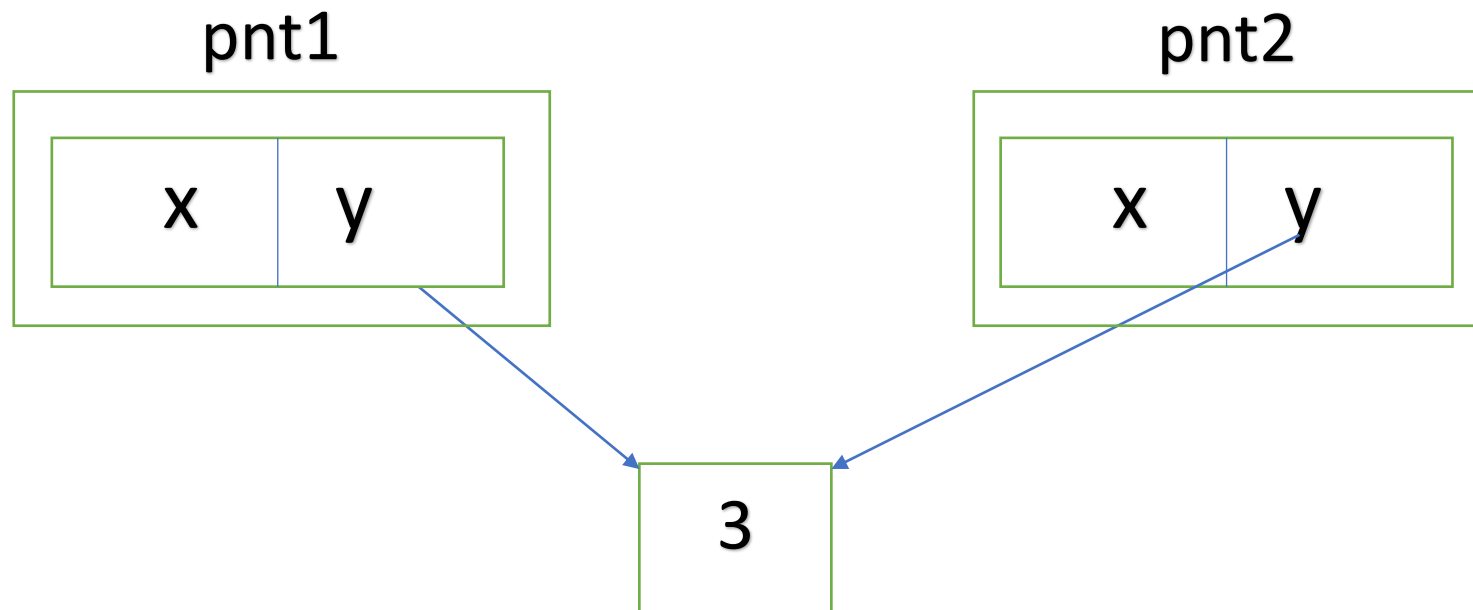
- Type of Copy constructor

1. **Default Copy Constructor** : When user/programmer does not provide the copy constructor in such case compiler provides the default copy constructor.
2. **User defined Copy Constructor** : When user/programmer provides the copy constructor.



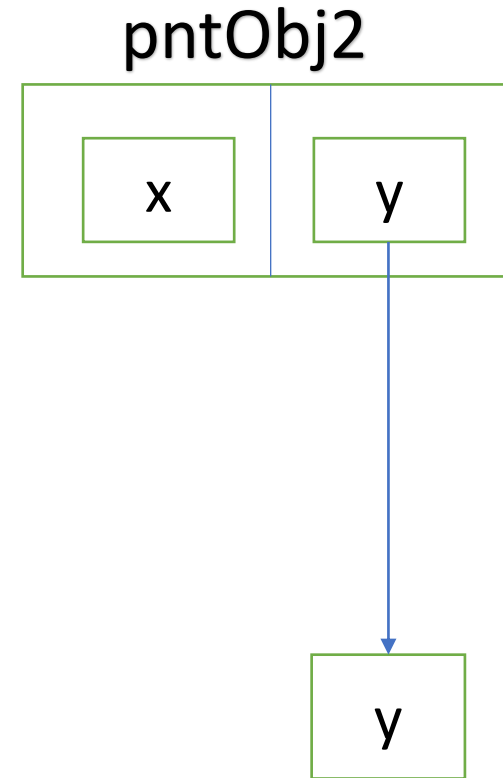
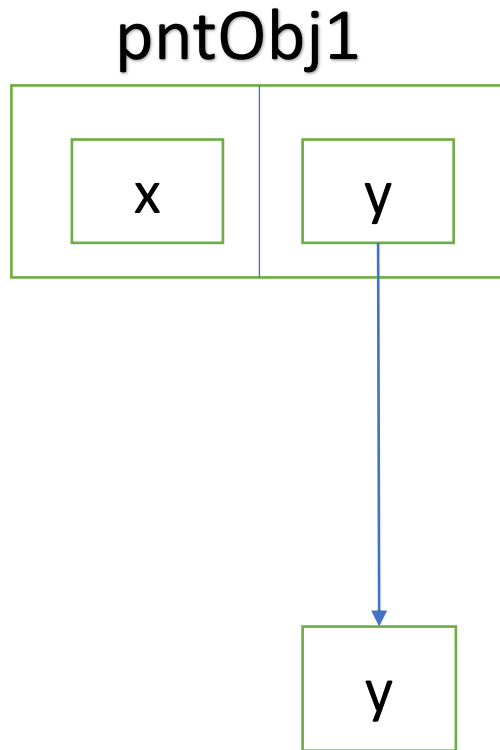
Shallow Copy

- Shallow copy is a copy of an object by coping data of all the **data members** it is also called as **bit-wise copy**.
- When programmer dose not provide copy constructor then compiler provides default copy constructor.
- The default constructor creates the exact copy or shallow copy of the existing object.



Deep Copy

- In case of deep copy memory is dynamically gets allocated for the copy and then only values are copied.
- Deep copy is also called as member wise copy.
- Deep copy requires user/programmer defined copy constructor.



Difference between malloc and new

new

new is an operator.

new returns a typecasted operator, so no need to do explicit typecasting.

We must mention the datatype while allocating the memory with new.

When memory is allocated with new, constructor gets called for the object.

malloc

malloc is a function.

malloc returns void pointer, malloc returns void pointer, cast it explicitly, before use.

malloc accepts only the exact no. of bytes required, so no need to mention datatype.

When memory gets allocated by malloc function, constructor function does not gets called.



Difference between malloc and new

new

Memory allocated by new is released by the operator delete.

Destructor is called when memory is released with delete.

To release memory for array syntax is
delete[]

In case new fails to allocate memory, it raises a Run time exception called as bad_alloc.

malloc

Memory allocated by malloc is released by function free.

Destructor is NOT called when the memory is released with free.

To release memory for array syntax is
free(ptr);

In case malloc fails to allocate memory it returns a NULL.



friend function

- If a function is declared as friend then **protected and private members of a class** can be used by the function.
- **friend keyword:** by using friend keyword compiler understands the function is a friend function.
- For accessing the private/protected data of a class you need to **give declaration of the friend function inside the class**.
- Friend function **do not get this pointer**.
- You can **declare main() as a friend function** but it is not recommended
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- Syntax:
 1. class class_name
 2. {
 //Class body
 //friend return_type function_name(parameter list);// you need to specify the function declaration with //friend keyword
 3. };



friend function

Example:

```
class Point
```

```
{
```

```
    int x,y;
```

```
    public:
```

```
        Point():x(10),y(20)
```

```
        {
```

```
        }
```

```
        friend void display(Point p);
```

```
}
```

```
void display(Point p)
```

```
{
```

```
    cout<<"x="<<p.x<<endl;
```

```
    cout<<"y="<<p.y<<endl;
```

```
}
```



Operator Overloading

- operator is token in C/C++.
- It is used to generate expression.
- operator is keyword in C++.
- Types of operator:
 - Unary operator (++,--,&!,~,sizeof())
 - Binary Operator (Arithmetic, relational, logical , bitwise, assignment)
 - Ternary operator (conditional)
- In C++, also we can not use operator with objects of user defined type directly.
- If we want to use operator with objects of user defined type then we should overload operator.
- To overload operator, we should define **operator function**.
- **We can define operator function using 2 ways:**
 - Using member function
 - Using non member function



Need Of Operator Overloading

- we **extend the meaning of the operator**.
- If we want to **use operator** with the **object of user defined type**, then we need to **overload operator**.
- To **overload operator**, we need to **define operator function**.
- In C++, **operator is a keyword**
 - Suppose we want to use **plus(+)** **operator** with objects then we need to define **operator+()** **function**.

We define operator function either inside class (as a member function) or outside class (as a non-member function).

Point pt1(10,20), pt2(30,40), pt3;

pt3 = pt1 + pt2; //pt3 = pt1.operator+(pt2); //using member function

//or

pt3 = pt1 + pt2; //pt3 = operator+(pt1, pt2); //using non member function



Operator Overloading

using member function

- **operator function must be member function**
- If we want to overload, binary operator using member function then **operator function should take only one parameter.**
 - Example : $c3 = c1 + c2$; //will be called as -
----- $c3 = c1.operator+(c2)$

Example :

```
Point operator+( Point &other ) //Member Function
```

```
{  
    Point temp;  
    temp.xPos = this->xPos + other.xPos;  
    temp.yPos = this->yPos + other.yPos;  
    return temp;  
}
```

using non member function

- **Operator function must be global function**
- If we want to overload binary operator using non member function then **operator function should take two parameters.**
 - **Example :** $c3 = c1 + c2$; //will be called as -
----- $c3 = operator+(c1,c2)$;

Example:

```
Point operator+( Point &pt1, Point &pt2 ) //Non Member  
Function
```

```
{  
    Point temp;  
    temp.xPos = pt1.xPos + pt2.xPos;  
    temp.yPos = pt1.yPos + pt2.yPos;  
    return temp;  
}
```



We can not overloading following operator using member as well as non member function:

1. dot/member selection operator(.)
2. Pointer to member selection operator(.*)
3. Scope resolution operator(::)
4. Ternary/conditional operator(? :)
5. sizeof() operator
6. typeid() operator
7. static_cast operator
8. dynamic_cast operator
9. const_cast operator
10. reinterpret_cast operator



We can not overload following operators using non member function

- Assignment operator(=)
- Subscript / Index operator([])
- Function Call operator[()]
- Arrow / Dereferencing operator(→)



Thank You

